

Vehicle-Type-Specific Gaussian Models for 3D Car Reconstruction and Classification

A Technical Walkthrough: From a Single Average Model to a Type-Aware Pipeline

Project Report

June 8, 2026

Abstract

This report documents the evolution of a 3D vehicle reconstruction pipeline. The original system used a *single*, type-agnostic statistical model that rendered every vehicle as a population-average shape. The redesigned system maintains *four* per-type Gaussian shape models and four parametric mesh templates (sedan, SUV, truck, van). Given a sparse 3D point cloud and predicted exterior dimensions, the new pipeline fits all four type-specific models, scores each by how well it explains the points, and both renders the best-fitting shape and classifies the vehicle by which model wins. We formalise the conditional-Gaussian shape model, trace exactly how the recovered parameters drive the mesh rendering, derive the selection score, and document the four engineering challenges whose solutions took classification accuracy from 1/3 to 3/3 on the labelled sample set. We also compare four interchangeable rendering back-ends — a single procedural template, four per-type parametric polygons, a rigidly-stretched photoreal `.glb`, and a parametric Kenney-kit reconstruction — and the trade-off they expose between visual realism and faithfulness to the predicted parameters.

Contents

1	Inputs and Parameterisation	2
2	The Original Single-Model Pipeline (Before)	2
2.1	The common Gaussian conditional model	2
2.2	Wireframe geometry and the height levels	3
2.3	Rigid fitting to the point cloud	3
2.4	Limitation of the original pipeline	4
3	The Multi-Model Redesign (Now)	4
3.1	Stage 1 — per-type Gaussian models	4
3.2	Stage 2 — per-type geometry templates	4
3.3	How the recovered parameters drive the rendering	5
3.4	Best-fit selection	5
4	Mathematical Formulation of the Selection Score	6
4.1	Robust (trimmed) geometric residual	6
4.2	Exterior-dimension prior	6
4.3	Combined score	6
5	Type-Selection Algorithm	6

6	Rendering Back-Ends: Four Display Strategies	6
6.1	<code>optimize_render_car.py</code> — the initial single template	7
6.2	<code>originalrender.py</code> — per-type parametric polygons (best so far)	8
6.3	<code>rendertooriginal.py</code> — a real photoreal <code>.glb</code>	8
6.4	<code>optimize_render_car_multimodel.py</code> — the Kenney Car Kit	9
6.5	Comparison	10
7	Challenges and Solutions	10
8	Results	10
9	Before vs. After: Summary	11
10	Limitations and Future Work	11

1 Inputs and Parameterisation

Each detected vehicle is described by two inputs:

1. A set of **predicted exterior dimensions** produced by an upstream detector.
2. A **sparse 3D point cloud** of 16–19 semantic keypoints.

The vehicle is modelled using 11 scalar parameters, partitioned into two groups: the *known* (observed) exterior dimensions b , and the *unknown* interior/proportion parameters a that must be predicted.

$$b = (OW, WB, OH, OL), \quad a = (A, C, D, E, F, G, TW). \quad (1)$$

Symbol	Meaning
OL, OW, OH	overall length, width, height
WB	wheelbase (conditioning only)
A	front bumper to base of windshield
C	side-window height
D	door height (below the window)
E	greenhouse (roof) width / inset
F, G	front and rear overhang (axle offsets)
TW	mean track width

Table 1: The 11 vehicle parameters.

Three quantities are derived deterministically from the above:

$$r_{\text{tire}} = OH - C - D, \quad w_{\text{tire}} = OW - TW, \quad h_{\text{belt}} = OH - C. \quad (2)$$

2 The Original Single-Model Pipeline (Before)

2.1 The common Gaussian conditional model

A *single* multivariate Gaussian was fitted over all vehicles in the pooled 2011–2023 dataset, jointly modelling the known dimensions b and the unknown proportions a :

$$\begin{pmatrix} a \\ b \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix}, \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix}\right). \quad (3)$$

The symbol $\sim \mathcal{N}(\mu, \Sigma)$ reads “is distributed as a multivariate Gaussian with mean vector μ and covariance matrix Σ .” The covariance blocks have the following sizes and meaning:

Block	Size	Meaning
Σ_{aa}	7×7	co-variation among hidden parameters
Σ_{bb}	4×4	co-variation among exterior dimensions
Σ_{ab}	7×4	cross-covariance (the key prediction block)
Σ_{ba}	4×7	transpose of Σ_{ab}

Building the covariance matrix. Given a dataset of n vehicles, each stacked into an 11-vector $x^{(k)} = (a^{(k)}, b^{(k)})$:

$$\mu = \frac{1}{n} \sum_{k=1}^n x^{(k)}, \quad \Sigma = \frac{1}{n-1} \sum_{k=1}^n (x^{(k)} - \mu)(x^{(k)} - \mu)^\top, \quad (4)$$

then partition Σ into the four blocks according to which indices belong to a and which to b .

Predicting the hidden parameters. Given an observed b , the missing parameters were predicted by the Gaussian *conditional mean*:

$$\hat{a} = \mathbb{E}[a | b] = \mu_a + \Sigma_{ab} \Sigma_{bb}^{-1} (b - \mu_b). \quad (5)$$

This is the exact, closed-form posterior mean when both variables are jointly Gaussian. Reading it term by term:

- $(b - \mu_b)$ — how far this vehicle’s exterior deviates from the average;
- Σ_{bb}^{-1} — normalises that deviation by the typical spread of exterior dimensions;
- Σ_{ab} — projects the normalised deviation into the hidden-parameter space using learned correlations;
- μ_a — the population baseline to which the data-driven correction is added.

2.2 Wireframe geometry and the height levels

The recovered parameters were turned into a fixed wireframe mesh (a lower body box, a greenhouse with sloped windshield and rear window, a flat roof, and four tire cylinders). The three defining vertical levels are:

$$y_{\text{sill}} = OH - C - D, \quad y_{\text{belt}} = OH - C, \quad y_{\text{roof}} = OH. \quad (6)$$

2.3 Rigid fitting to the point cloud

The mesh *shape* was fixed; only its *pose* was optimised. Let $P = \{p_i\}_{i=1}^N$ be the (centred, cm-scaled) point cloud and S the mesh surface. With a single global translation $t \in \mathbb{R}^3$ and a heading rotation $R(\theta)$ about the vertical axis, each point is mapped into the model frame:

$$q_i = R(\theta)^\top (p_i - t), \quad (7)$$

and its unsigned distance to the nearest surface element is computed:

$$d_i(t, \theta) = \min_{s \in S} \text{dist} \left(R(\theta)^\top (p_i - t), s \right). \quad (8)$$

For a quad face the distance is the in-plane projection distance when the projection falls inside the quad, and the nearest-edge distance otherwise. The pose minimises the weighted mean squared distance,

$$\mathcal{L}(t, \theta) = \frac{1}{N} \sum_{i=1}^N w_i d_i(t, \theta)^2, \quad (9)$$

using 50 iterations of RMSProp (autograd through the distance functions), with w_i up-weighting the tire term for ground-level points.

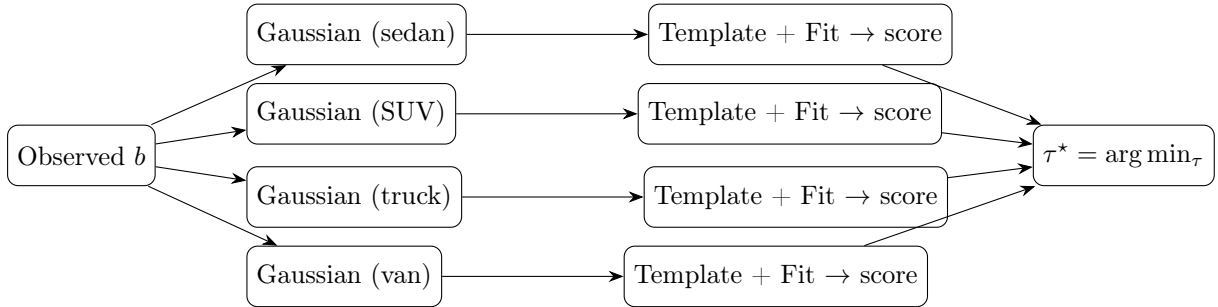
Remark 1 (Translation vs. rotation). The translation t is a *single global vector* subtracted from every point identically; it is order- and distance-preserving and can never turn a front point into a rear point. Only the rotation $R(\theta)$ can flip orientation. A 180° heading error causes front keypoints to be matched against rear mesh surfaces in Eq. (8) — the root of Challenge 2.

2.4 Limitation of the original pipeline

Because a single Gaussian and a single template were used, the rendered shape was always the population average — a fastback/SUV-like silhouette. The system had no notion of vehicle type and could represent neither a pickup’s bed nor a sedan’s trunk.

3 The Multi-Model Redesign (Now)

The redesign keeps the same conditional-Gaussian machinery but *instantiates it per type* and adds a model-selection step. No new mathematical machinery is introduced.



3.1 Stage 1 — per-type Gaussian models

Four separate Gaussians are trained, one per category $\tau \in \{\text{sedan, suv, truck, van}\}$, each only on that category’s rows. Conditioning on the same observed b now yields a *type-specific* prediction:

$$\hat{a}^{(\tau)} = \mu_a^{(\tau)} + \Sigma_{ab}^{(\tau)} (\Sigma_{bb}^{(\tau)})^{-1} (b - \mu_b^{(\tau)}). \quad (10)$$

The same b produces four geometrically distinct predictions because each model learned different average proportions and correlations from its own data (e.g. a truck Gaussian predicts a short greenhouse and long bed; a sedan Gaussian predicts a lower roofline and a trunk gap).

3.2 Stage 2 — per-type geometry templates

The single template is replaced by four, all built from one shared “model-car” recipe (the Sketch-up / Model-Car-Vertices construction from the design doc). Every type reuses an identical *tapered nose* (an inset grille panel chamfered out to full width, faces 1–5), an identical *lower body* (the doors, beltline down to sill) and an identical *underbody*; only the structure *above the beltline* changes per type. This keeps the four meshes watertight and mutually consistent while

letting each express its own greenhouse, trunk or bed. The greenhouse is laterally inset by $(OW - E)/2$, so the recovered E narrows the cabin directly.

Type	Structure above the beltline	Quads
SUV	two-box; inset greenhouse, rear glass straight to the beltline	14
Van	two-box, taller; upright windshield, roof carried to the tail	14
Sedan	three-box; inset cabin, sloped rear glass, lower full-width trunk deck (with C-pillar “sail” quads)	19
Truck	full-width cab + open bed (bed floor, side rails, tailgate)	18

Table 2: The four parametric templates (`car_models_core_new`). The fitting loss is generic over the quad list, so differing face counts need no special casing.

3.3 How the recovered parameters drive the rendering

Every mesh vertex is a deterministic function of the 11 parameters. The key insight is that the outer bounding box comes from b , but *almost everything inside it* comes from the predicted $\hat{a}^{(\tau)}$ — which is precisely why four different $\hat{a}^{(\tau)}$ over the same b yield four discriminable shapes.

Param	Source	Used to build
OL	b	body length, all z -profile positions, rear bumper
OW	b	body width, tire width ($OW - TW$)
OH	b	y_{roof} ; feeds y_{belt} , y_{sill} ; tire radius
WB	b	rear axle position ($F + WB$)
A	$\hat{a}^{(\tau)}$	windshield-base z , roofline start $A + \frac{2}{10}(OL - A)$
C	$\hat{a}^{(\tau)}$	$y_{\text{belt}} = OH - C$, $y_{\text{sill}} = OH - C - D$, window height, tire radius
D	$\hat{a}^{(\tau)}$	$y_{\text{sill}} = OH - C - D$, tire radius
E	$\hat{a}^{(\tau)}$	greenhouse lateral inset (<code>off</code>), roof narrowing
F	$\hat{a}^{(\tau)}$	front axle z -position, roofline anchor
G	$\hat{a}^{(\tau)}$	rear overhang, rear-axle confirmation $G = OL - F - WB$
TW	$\hat{a}^{(\tau)}$	tire lateral positions ($\pm TW/2$), tire width ($OW - TW$)

Table 3: Parameter-to-geometry map. b sets the outer box; $\hat{a}^{(\tau)}$ sets the interior.

3.4 Best-fit selection

All four type models are built and fitted; the type whose fitted shape best explains the points is both rendered and taken as the predicted class:

$$\tau^* = \arg \min_{\tau} \text{score}(\tau). \quad (11)$$

Because the exterior box (OW, OH, OL, WB) is identical across the four candidates, the comparison isolates the internal silhouette — exactly where the categories differ.

4 Mathematical Formulation of the Selection Score

The score combines a robust geometric residual with a statistical prior on the exterior dimensions.

4.1 Robust (trimmed) geometric residual

Let $B \subseteq P$ be the body points (above a small height threshold) and $d_{(1)} \leq d_{(2)} \leq \dots \leq d_{(M)}$ the sorted point-surface distances at the fitted pose. Discarding the worst fraction ρ of points yields:

$$R_{\text{trim}}(\tau) = \frac{1}{K} \sum_{i=1}^K d_{(i)}^2, \quad K = \lfloor (1 - \rho) M \rfloor, \quad \rho = 0.2. \quad (12)$$

This is the mean squared distance of the 80% best-fitting body points — an outlier-resistant estimator that removes accessory keypoints (cargo racks, antennas) that conform to no template.

4.2 Exterior-dimension prior

Each type’s Gaussian induces a *marginal* over the exterior dimensions, obtained by integrating out the unobserved a :

$$b \mid \tau \sim \mathcal{N}(\mu_b^{(\tau)}, \Sigma_{bb}^{(\tau)}). \quad (13)$$

The marginal is the correct likelihood to use because a is never observed, and substituting the predicted $\hat{a}^{(\tau)}$ would be circular (it already assumes type τ). Its negative log-likelihood (dropping the type-independent $\frac{d}{2} \log 2\pi$ constant) is:

$$\text{NLL}(\tau) = \underbrace{\frac{1}{2} (b - \mu_b^{(\tau)})^\top (\Sigma_{bb}^{(\tau)})^{-1} (b - \mu_b^{(\tau)})}_{\text{Mahalanobis distance}} + \underbrace{\frac{1}{2} \log |\Sigma_{bb}^{(\tau)}|}_{\text{log-determinant}}. \quad (14)$$

- **Mahalanobis term:** the statistical distance of b from type τ ’s typical dimensions, normalised by the type’s spread. A 5.8m vehicle scores low against the truck mean and devastatingly high against the sedan mean.
- **Log-determinant term:** a normalisation that penalises broad, diffuse distributions. Without it, a type with huge variance would win by accepting almost any vehicle; the log-det makes a tight, specific distribution score lower when the dimensions genuinely match.

4.3 Combined score

$$\boxed{\text{score}(\tau) = R_{\text{trim}}(\tau) + \lambda \text{NLL}(\tau)} \quad (15)$$

with λ balancing geometric fit against the dimensional prior. The first term answers “*does this shape match the points?*”; the second answers “*are these dimensions typical of this type?*” Structurally this is a Bayesian model-selection criterion: R_{trim} acts as the likelihood, NLL as the prior, and τ^* is the MAP decision.

5 Type-Selection Algorithm

6 Rendering Back-Ends: Four Display Strategies

The type decision and the fitted pose are produced once by the selection core (`car_models_core`); *what is drawn at that pose* is a separate choice. Four interchangeable renderers were built, trading off visual realism against how many of the eleven predicted parameters actually deform the displayed mesh. This is the central tension: **the more photoreal the display, the fewer of the predicted parameters it can honour.**

Algorithm 1: Type selection for one vehicle

Require: point cloud P , exterior dims $b = (OW, WB, OH, OL)$, heading guess θ_0
for $\tau \in \{\text{sedan, suv, truck, van}\}$ **do**
 $\hat{a}^{(\tau)} \leftarrow \mu_a^{(\tau)} + \Sigma_{ab}^{(\tau)} (\Sigma_{bb}^{(\tau)})^{-1} (b - \mu_b^{(\tau)})$ ▷ Eq. (10)
 build mesh $S^{(\tau)}$ from $(\hat{a}^{(\tau)}, b)$
 for $\delta \in \{0^\circ, 180^\circ\}$ **do** ▷ heading multi-start
 $(t, \theta) \leftarrow \arg \min \mathcal{L}$ from $\theta_0 + \delta$ ▷ Eq. (9)
 keep the start with the smaller R_{trim} ▷ Eq. (12)
 end for
 score(τ) $\leftarrow R_{\text{trim}}(\tau) + \lambda \text{NLL}(\tau)$ ▷ Eqs. (14)–(15)
end for
return $\tau^* = \arg \min_{\tau} \text{score}(\tau)$

Shared technology stack. All four use the same plumbing: a **PyQt5** `QOpenGLWidget` for the window and GL context, **PyOpenGL** (`OpenGL.GL/GLU/GLUT`) for immediate-mode drawing, **NumPy/pandas** for data, and the **PyTorch** RMSProp fit inside the core. The two mesh-based back-ends additionally use **trimesh** to load `.gltf` (binary glTF) scene graphs, bake the node transforms, and extract per-vertex colours from the kit’s palette texture; the baked triangle soup is cached in an OpenGL display list (`glGenLists/glCallList`).

6.1 `optimize_render_car.py` — the initial single template

The original pipeline. A *single* pooled Gaussian (Eq. (5)) predicts the parameters and a *single* hard-coded 22-vertex mesh is built and fitted (PyTorch autograd through the quad-distance functions; a Matplotlib loss-vs-iteration plot is emitted). The same silhouette is used for *every* vehicle — there is no type concept and no selection.

- + Simplest; exercises all 11 parameters on one shape; the foundation the rest of the work builds on.
- One shape for all vehicles: always a fastback/SUV-like silhouette. It can represent neither a pickup’s bed nor a sedan’s trunk, and performs no classification.

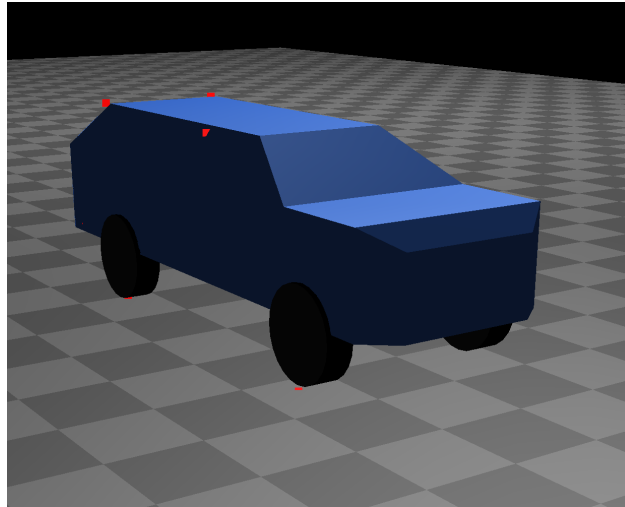


Figure 1: `optimize_render_car.py` on `points0`: one fixed template fitted to the keypoints (red). The same mesh would be used for a van or a pickup.

6.2 originalrender.py — per-type parametric polygons (best so far)

The current canonical back-end (uses `car_models_core_new`). It fits the four type-specific parametric templates of Section 3, keeps the winner, and draws *that parametric mesh directly*. Because every vertex is a function of the parameters (Table 3), the displayed body genuinely flexes to all eleven recovered values — length, width, height, hood, beltline, door height, greenhouse inset, axle positions and track all reshape the mesh.

- + Uses *all 11* parameters to resize *and* reshape; four distinct, watertight type silhouettes; the display *is* the thing being scored, so what you see is exactly what was fitted.
- Low-fidelity “boxes + cylinders” look: faceted, no curves or detailing — not a convincingly vehicle-like structure.

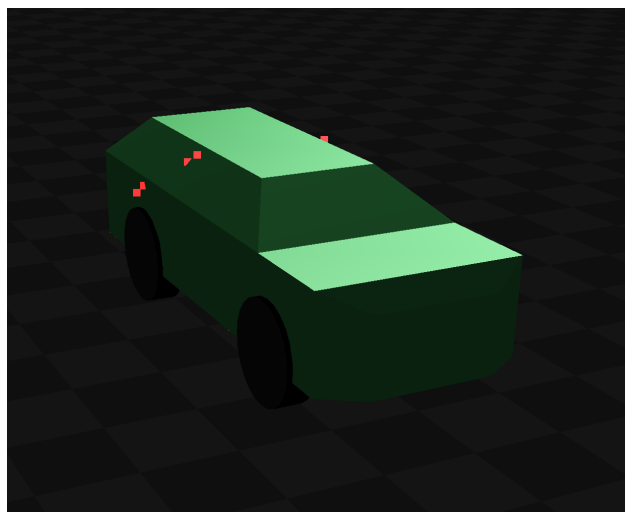


Figure 2: `originalrender.py` on `points0`: the winning SUV template, fully parametric. The mesh tracks every predicted dimension but looks blocky.

6.3 rendertooriginal.py — a real photoreal .glb

Selection runs as usual, but the winner is drawn as a downloaded, photoreal production model (`models/<type>.glb`, e.g. a BMW X5) via `vehicle_meshes.render_arrays()`. The stock mesh is *rigidly* stretched so its bounding box matches the predicted $OW \times OH \times OL$ — nothing else moves.

- + By far the most realistic, immediately recognisable vehicle on screen.
- Only the exterior box (OW , OH , OL) deforms it; the seven interior/derived parameters cannot reshape it, so it can never be made to match the *observed* car’s true proportions (wheelbase, overhangs, greenhouse). It is a stretched stock model, not a reconstruction.

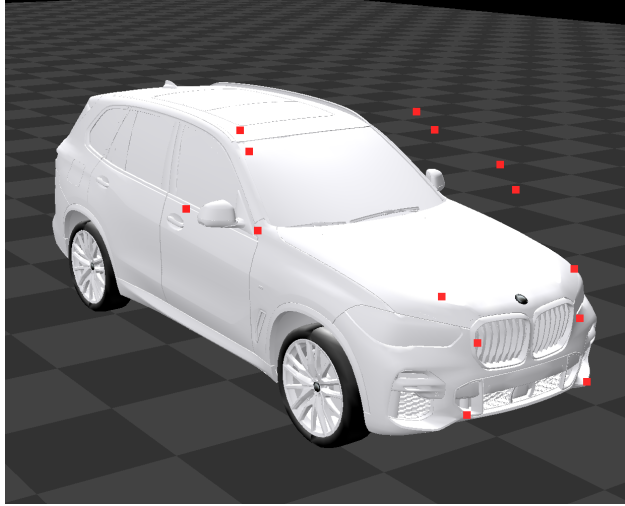


Figure 3: `rendertooriginal.py`: a photoreal `.glb` stretched to the predicted bounding box. Realistic, but blind to the seven interior parameters.

6.4 `optimize_render_car_multimodel.py` — the Kenney Car Kit

A middle ground. The winner is drawn as a low-poly **Kenney Car Kit** (CC0) model via `vehicle_meshes.reconstruct()`: the body is scaled to $OW \times OH \times OL$, then each *named* wheel group is relocated to the derived front/rear axles ($OL/2 - F$, $-OL/2 + G$), snapped to the track half-width $TW/2$, and resized to r_{tire} .

- + Looks much more like a real vehicle than the procedural box; wheels actually move to the predicted wheelbase, track and radius.
- Honours only 6 of 11 parameters (OW , OH , OL , F , G , TW , plus r_{tire}); the body-shape parameters A, C, D, E are *not* applied (the body is a rigidly scaled stock mesh), so the greenhouse/trunk/bed proportions stay fixed and the prediction is under-used.

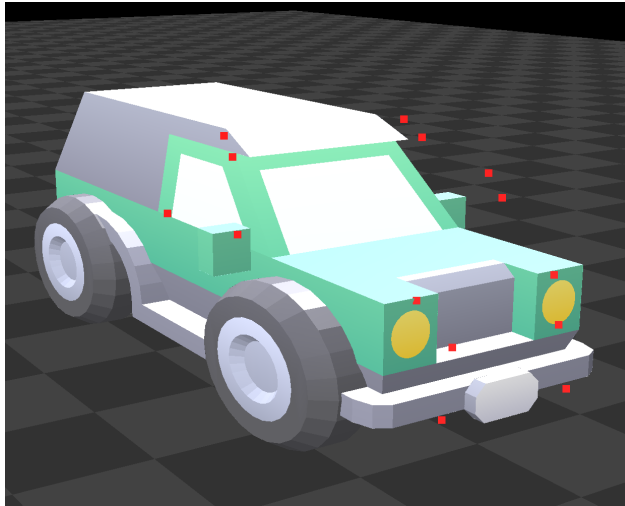


Figure 4: `optimize_render_car_multimodel.py`: a Kenney low-poly model with wheels relocated to the derived wheelbase/track. Vehicle-like, but the body shape (A, C, D, E) is not parametric.

Renderer	Display mesh	Params	Realism	Type-aware
optimize_render_car1	procedural template	11 (one shape)	low	no
originalrender	4 procedural templates	11	low	yes (4)
rendertooriginal	photoreal .glb	3 (OW, OH, OL)	high	selection only
multimodel (Kenney)	low-poly .glb	6 ($+r_{\text{tire}}$)	med-high	selection only

Table 4: The four back-ends. “Params” is how many of the 11 predicted values actually deform the drawn mesh. **originalrender** maximises faithfulness to the prediction; the .glb back-ends maximise visual realism. The ideal — a fully parametric *and* photoreal mesh — remains future work.

6.5 Comparison

7 Challenges and Solutions

Challenge	Root cause	Solution
1. Tire-term domination	tire geometry near-identical across types, carrying no class signal	score on the body-only residual; reduce the tire weight
2. Heading (180°) ambiguity	non-convex loss trapped the optimiser in a reversed orientation	multi-start from θ_0 and $\theta_0 + 180^\circ$, keep lower residual
3. Sparse key-points / accessories	a cargo rack inflated the truck residual, misclassifying it as sedan	trimmed residual: discard worst $\rho = 0.2$ of points
4. Weak geometric discrimination	shared exterior box + sparse points leave types too similar	exterior-dimension prior $\text{NLL}(\tau)$ with $\lambda = 16$

Table 5: The four engineering challenges and their fixes.

8 Results

Three labelled point clouds were used for validation: **points0** (an SUV), **points1** (a van) and **points2** (a pickup). Accuracy improved as each fix was added.

The combined scores below are from the canonical back-end (**originalrender.py** + **car_models_core_new**). Every cloud is classified correctly, with the score broken into its two terms $\text{score}(\tau) = R_{\text{trim}}(\tau) + \lambda \text{NLL}(\tau)$, $\lambda = 16$.

The .glb and Kenney back-ends (**rendertooriginal.py**, **optimize_render_car_multimodel.py**) share an *earlier* selection core (**car_models_core**); their selection therefore agrees on the winner but with slightly different residuals from the older templates — e.g. on **points0** both score SUV 192.2 / sedan 216.5 / truck 282.4 / van 310.3, the same SUV winner.

Configuration	SUV	Van	Pickup	Accuracy
Raw residual ($\times 5$ tire weight)	four-way tie ($\sim 8 \times 10^4$)			—
Body residual + 180° multi-start	truck	van	sedan	1/3
+ trimmed residual ($\rho = 0.2$)	suv	van	suv	2/3
+ exterior prior ($\lambda = 16$)	suv	van	truck	3/3

Table 6: Classification accuracy as each fix is introduced.

Cloud (truth)	sedan	suv	truck	van
points0 (SUV)	233.6	206.8	302.9	286.7
points1 (Van)	452.1	458.3	496.6	224.4
points2 (Pickup)	302.9	419.3	272.9	293.4

Table 7: Final combined scores $\text{score}(\tau)$ (Eq. (15); lower is better, winner in bold). **3/3** correct.

9 Before vs. After: Summary

The core idea. One ambiguous reconstruction problem was decomposed into four simpler “does this type fit?” problems, each scored by geometry (does the shape match the points?) plus statistics (are the dimensions typical of this type?). Picking the winner yields reconstruction and classification simultaneously — using no new mathematics beyond the conditional Gaussian, fitting loss, and RMSProp optimiser already present in the original pipeline.

10 Limitations and Future Work

- **Prior weight calibration.** λ was tuned on three vehicles; any $\lambda \in [12, 50]$ gives 3/3 here, but it should be recalibrated against the full category-labelled dataset.
- **Keypoint correspondence.** The most robust upgrade is to exploit keypoint labels (e.g. the four wheels) and fit with known correspondences rather than nearest-surface distances — sidestepping the accessory-outlier problem entirely.
- **Template refinement.** The per-type silhouette constants (cab length, bed/trunk height, roofline fractions) are visual knobs to be tuned against rendered overlays.
- **Pose robustness.** Trimming is currently applied only to the selection residual; applying it inside the pose optimiser could further reduce outlier influence on alignment.

Cloud	Type	R_{trim}	NLL	16 NLL	score
points0	suv ★	27.72	11.20	179.2	206.84
	sedan	25.51	13.00	208.1	233.56
	van	26.53	16.26	260.2	286.71
	truck	45.87	16.07	257.0	302.91
points1	van ★	33.00	11.96	191.4	224.41
	sedan	216.52	14.72	235.6	452.10
	suv	128.81	20.60	329.5	458.33
	truck	211.56	17.81	284.9	496.55
points2	truck ★	99.21	10.85	173.6	272.86
	van	81.12	13.27	212.3	293.43
	sedan	83.52	13.71	219.4	302.94
	suv	59.01	22.52	360.3	419.30

Table 8: Per-cloud selection breakdown (★ = winner). The pickup (points2) is the closest call: geometrically the box-like types fit slightly better (R_{trim} of van/sedan/suv < truck), but the dimensional prior NLL decisively favours the truck, which wins the combined score. A truck-only template tweak (raking the windshield, `roof_front` 0.18 $\rightarrow$ 0.28) lowered the truck residual from 127.8 to 99.2 and secured the ~ 20 -point margin shown.

Aspect	Before (single model)	After (multi-model)
Gaussian	one, pooled over all vehicles	four, one per type τ
Prediction	$\hat{a} = \mu_a + \Sigma_{ab} \Sigma_{bb}^{-1} (b - \mu_b)$	same formula applied 4 $\times$ with type-specific blocks
Template	one fixed fastback silhouette	four distinct silhouettes
Fitting	$\mathcal{L}$ minimised once	$\mathcal{L}$ minimised 4 $\times$ independently
Score	raw fitting loss only	$R_{\text{trim}} + \lambda \text{NLL}$ (geometry + prior)
Output	one average shape	best shape + vehicle class for free